

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	V.LASYA	Reg. No.:	192524186
Section / Batch:	B.TECH AI&DS	Date:	31-08-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONSTARINT BASED PROBLEM SOLVING

C05_AT1_Constraint-Based Problem Solving

[Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1

Backtracking and Tractability
10 marks

Q2

Backtracking and Tractability
10 marks

1/2 answered

Q1: Backtracking and Tractability

10 marks

Knight's Tour Problem (Backtracking / Constraint Satisfaction Problem)

A knight must visit every square of a chessboard exactly once. Clearly define constraints related to valid knight moves and non-repetition. Analyze how these constraints reduce possible moves at each step. Develop a backtracking-based solution to explore paths. Apply pruning to eliminate dead-end paths. Justify how your solution ensures completeness and discuss complexity.

Your Code

```
def solve_knights_tour(n):  
    """Solve the Knight's Tour problem using backtracking.  
    n: Integer board size (e.g., 5 or 8).  
    Returns: List of moves or None if no solution exists.  
    """  
    # 1. Initialize the board with -1 (representing unvisited)  
    board = [[-1 for _ in range(n)] for _ in range(n)]  
    # 2. Define the 8 possible moves a knight can make  
    moves_x = [2, 1, -1, -2, -2, -1, 1, 2]  
    moves_y = [1, 2, 2, 1, -1, -2, -2, -1]  
    # 3. Backtracking function  
    def backtrack(x, y, count):  
        # Base Case: If all squares are visited, we are done!  
        if count == n * n:  
            return True  
        # Try all 8 possible moves  
        for i in range(8):  
            next_x = x + moves_x[i]  
            next_y = y + moves_y[i]  
            # Check if the move is valid (within bounds and unvisited)  
            if 0 <= next_x < n and 0 <= next_y < n and board[next_x][next_y] == -1:  
                # Recursive step: try to find the next move  
                board[next_x][next_y] = count  
                if backtrack(next_x, next_y, count + 1):  
                    return True  
            # BACKTRACKING: If move doesn't work, undo it (reset to -1)  
            board[next_x][next_y] = -1  
        return False  
    # 4. Run the solver  
    return backtrack(0, 0, 0)
```

Input

5

Output

```
Enter board size (e.g., 5 or 8):  
Success! Here is the Knight's  
Tour path:  
0  0 14  8 20  
14  8 18  4 10
```

SIMATS Online Programming Lab
DATA PROGRAMMING

Random Login
Not Logged In

CO5_AT1_Constraint Based Problem Solving
Back

Language: Python C++ Java

QUESTIONS

Q1

Backtracking and Tractability

10 marks

Q2

Backtracking and Tractability

10 marks

10 answered

Backtracking and Tractability
10 marks

Boolean Formula Minimization (TSP-Hard Problem)
 Simplify a Boolean expression to its minimal equivalent form. Clearly define constraints related to logical equivalence and reduction rules. Analyze how constraints influence simplification. Develop a backtracking-based solution to explore combinations. Apply pruning to eliminate redundant expressions. Justify correctness and discuss TSP-Hard nature.

Your Code

```

import itertools
#
def eval_exp(expr, var_map, assignments):
    # returns True if correct, each node is a tuple like ('A', 10) means
    # the term on term:
    # A = True
    # A is in term:
    # if not assignments['A']:
    #     return False
    # else:
    #     return True
    # if not:
    #     return False
    # return True
    return False

def evaluate(term, term, var_map):
    # for term on (term, term, term, term, term, term, term, term)
    # assignments = var_map(term, term)
    # if eval_exp(term, var_map, assignments) != eval_exp(term,
    # return False
    return True

def min_term(term, var_map):
    # a top smaller number from backtracking idea by return search
    # a = len(term)
    # for e in range(1, a + 1):
    #     for subset in itertools.combinations(term, e):
    #         if evaluate(subset, term, var_map):
    #             return len(subset)
    # return None

# a ----- Easy input / output -----
# a input from user
# a term: A, B, C
# a term: A, B, A, B, C, B, C
# a where each node is "node" (A,B,C) node are separated by ,
#
print("Enter variables separated by space (example: A B C):")
var_map = input().split()
#
print("Enter term separated by comma (example: A B, A C, B C)")
a = input().split(',')
#
var_map = {v: 0 for v in a.split(',') if v}
term = []
for e in a:
    term.append(e)
    
```

Input

```

A B C
A, B, A, C
    
```

Output

```

Enter variables separated by
space (example: A B C):
Enter term separated by
comma (example: A B, A C, B C)
Expected output: 1120 (A, B, C)
    
```

Code of Conduct

I V.LASYA(192524186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

V.lasya

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Applicatio n of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justificati on	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Questi on No.	Problem Understandi ng (25%)	Constraint Analysis (25%)	Solution Developm ent (25%)	Applicati on of Concepts (15%)	Justificati on (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____